

When Agent Success Is Not Agent Skill: A Causal Benchmark for Tool-Using LLM Agents

CausalAgentBench Contributors

Abstract

Tool-using language agents are increasingly evaluated by final task success, but such scores conflate planning, tool selection, memory use, observation interpretation, recovery, and stopping behavior. We introduce CAUSALAGENTBENCH, an interventional benchmark that pairs clean agent tasks with controlled perturbations targeting individual skill components. In the planned final study across [N] tasks, [M] intervention instances, and [K] agents, we will test whether clean success overestimates robust competence by [X] points on average, and whether model rankings under clean success correlate only [rho] with rankings under causal robustness. We propose Agent Causal Robustness Score (ACRS) and a suite of trajectory-level diagnostics designed to expose failures hidden by final-answer scoring. CAUSALAGENTBENCH provides a reproducible evaluation framework for measuring not only whether agents succeed, but which agentic skills survive under controlled environmental stress.

1 Introduction

Tool-using language agents are now commonly evaluated by whether they complete a task. This final-success framing is attractive because it is simple, task-facing, and easy to compare across systems. Yet final success is a compressed outcome: it does not identify whether the agent planned well, chose the right tool, supplied valid tool arguments, interpreted observations correctly, verified memory, recovered from errors, or stopped for the right reason. This paper targets that measurement problem.

Consider an agent that solves a clean travel-planning task by calling a hotel search tool and returning an acceptable recommendation. The same high-level user goal may become much more diagnostic if the search tool is removed, returns a partial result, or conflicts with a stored memory. A robust agent should notice the changed condition, use alternatives when available, surface uncertainty when evidence is weak, and avoid declaring success too early. A final-answer score alone can miss these distinctions. This motivates **Claim C1**: clean task success may overestimate robust competence.

CAUSALAGENTBENCH formalizes this concern as a paired interventional benchmark. Each base task has a clean condition and one or more intervention conditions that preserve the high-level user goal while perturbing a targeted factor such as tool availability, tool reliability, memory correctness, observation consistency, or premature success signals. The benchmark then scores both final answers and trajectories, producing component diagnostics for tool use, recovery, contradiction handling, memory verification, stopping behavior, and trajectory faithfulness. These diagnostics support the planned tests in **Claim C2–Claim C8**.

Our contributions are:

1. We formalize interventional evaluation for tool-using agents, separating clean task success from robustness under targeted perturbations.
2. We introduce CAUSALAGENTBENCH, a benchmark of paired clean and intervention instances spanning [domains].
3. We propose Agent Causal Robustness Score (ACRS) and trajectory diagnostics for tool use, recovery, contradiction handling, memory verification, and stopping behavior.
4. We provide an experimental protocol to evaluate [agents/models] and test [main finding placeholder].
5. We release code, benchmark generation tools, documentation, and reproducibility assets.

The empirical findings in this draft are placeholders. The repository contains deterministic smoke and development runs for engineering validation, but the claim ledger currently marks the main scientific claims as planned rather than supported. Before submission, each numerical statement in this section must be replaced by results linked to specific configs, run directories, and claim-ledger rows.

2 Related Work

Agent and tool-use benchmarks. Existing agent benchmarks evaluate web navigation, software engineering, tool use, planning, and multi-step task completion. The related-work tracker in `docs/RELATED_WORK_TRACKER.md` lists the papers and benchmarks that must be reviewed before submission. This draft intentionally uses TODO bibliography entries rather than invented citations [1, 3, 4, 2].

Trajectory-level evaluation. Several evaluation efforts inspect intermediate reasoning, actions, or traces rather than only final answers. CAUSALAGENTBENCH builds on that idea by treating trajectory metrics as diagnostics for separable agent skills rather than as a replacement for final task success. The planned evidence for this distinction is tracked by **Claim C3**.

Robustness and causal/intervention-based evaluation. Robustness benchmarks often vary input phrasing, distribution, adversarial examples, or environment state. CAUSALAGENTBENCH focuses on paired task instances where a targeted factor is changed while the high-level user goal is held fixed. The intended contribution is not merely harder tasks, but a causal comparison between clean and intervened conditions. This is the basis of **Claim C1**, **Claim C2**, and **Claim C10**.

Why paired interventions are scientifically different. The central distinction is the estimand. A standard perturbation benchmark asks whether an agent’s score changes on a harder or shifted dataset. CAUSALAGENTBENCH asks whether a specific agent skill survives when a named factor is intervened on while the user goal and base task are held fixed. This does not automatically prove causality; it creates an auditable design where intervention validity can be inspected and challenged. The final paper must show that this design yields information not captured by aggregate success alone.

LLM-as-judge and human validation. Final-answer and trajectory labels can be exact, heuristic, human-annotated, or model-judged. The default CAUSALAGENTBENCHsmoke path uses deterministic scoring only. If LLM-as-judge scoring is added, the paper must report grader prompts, model versions, agreement rates, and failure modes, and must not treat judge labels as ground truth without validation. Human validation is planned under **Claim C3** and **Claim C10**.

3 Benchmark Design

CAUSALAGENTBENCH is organized around base tasks, clean instances, intervention instances, trajectories, and scores. A base task specifies a user instruction, success criteria, required information, forbidden assumptions, available tools, hidden ground truth for scoring, an optional gold tool sequence, maximum step budget, domain, difficulty, and metadata. A clean instance exposes the base task without perturbation. An intervention instance references the same base task and applies a controlled change.

Task domains. The planned benchmark spans travel planning, calendar/email workflow, file and spreadsheet question answering, shopping/comparison, research assistant tasks, policy/compliance tasks, coding/debugging tasks, and multi-hop operational planning. The released generator currently supports these synthetic domains with deterministic templates. The final paper should report the exact domain counts from the submitted dataset in Table 1.

Table 1: Benchmark statistics. Placeholder until the final dataset is frozen. The generated table template is `tables/table1.benchmark_statistics.*`.

Statistic	Value
Base tasks	[N]
Intervention instances	[M]
Domains	[domains]
Difficulty levels	[difficulty mix]
Average required tools	[avg]
Average maximum steps	[avg]

Simulated tools. The default environment uses deterministic local tools: database search, policy lookup, calendar checking, file reading, spreadsheet querying, price calculation, option comparison, email-draft creation, booking stubs, and fact verification. No default experiment uses live web access, real email sending, real bookings, private data, or paid model calls.

Why start synthetic. Live web or enterprise tasks are more realistic, but they make causal attribution brittle: tool outputs drift, access permissions change, websites update, and hidden state is hard to audit. CAUSALAGENTBENCH starts with deterministic simulation because the first scientific question is not whether an agent can operate a particular website, but whether targeted changes to tool availability, memory, observations, and stopping signals expose separable failures. This is a tradeoff. The benchmark needs later external-validity work on messier environments.

Artifacts and validation. Benchmark data is stored as JSONL and validated with Pydantic schemas. Generated datasets include `base_tasks.jsonl`, `interventions.jsonl`, `instances.jsonl`, `generation_report.json`, and `quality_report.md`. Runs save trajectories, errors, score records, aggregate summaries, config hashes, and metadata. The reproducibility goal is tracked by **Claim C9**.

4 Interventional Framework

For each base task b , CAUSALAGENTBENCH constructs a clean instance x_b^0 and intervention instances x_b^j for intervention families j . Each intervention is intended to change one targeted factor while keeping the user’s high-level goal fixed. The intervention metadata records the family, changed factor, expected robust behavior, severity, patch details, and whether the final answer is expected to change.

The current intervention families are tool removal, tool failure, tool corruption, irrelevant tools, memory corruption, observation conflict, ambiguous instruction, long-horizon dependency, premature success signal, and distractor evidence. These families target the skill components described in the claim ledger: recovery under failures (**Claim C2**, **Claim C5**), contradiction handling (**Claim C3**, **Claim C8**), memory verification (**Claim C2**, **Claim C3**), tool overuse (**Claim C7**), and stopping behavior (**Claim C8**).

The causal interpretation depends on intervention validity. A tool-failure condition should primarily affect tool reliability, not silently alter the user’s objective or the correct answer unless explicitly specified. A memory-corruption condition should make memory less trustworthy while still allowing verification. The benchmark includes quality checks that flag interventions likely to change multiple factors, but expert or human audit remains necessary. This remaining risk is tracked by **Claim C10**.

5 Metrics

Final task success. The benchmark reports binary and partial final-answer success using deterministic heuristic scoring in the default package. Human or model-based scoring can be added later, but any such scorer must be separately validated.

Trajectory diagnostics. Trajectory metrics include required-tool recall, tool precision, unnecessary-tool-call rate, missing required tools, invalid tool calls, argument validity, argument errors, recovery from tool errors, repeated failed calls, contradiction detection and resolution, memory use and verification, blind trust in corrupted memory, premature stopping, maximum-step failure, correct stopping, trajectory efficiency, and trajectory faithfulness. These metrics support **Claim C3**, **Claim C5**, **Claim C7**, and **Claim C8**.

Agent Causal Robustness Score. For an agent a , let $S_{\text{clean}}(a)$ denote clean-condition success rate and $S_{\text{int}}(a)$ denote intervention-condition success rate. The initial robustness score is

$$\text{ACRS}(a) = \frac{S_{\text{int}}(a)}{S_{\text{clean}}(a)}.$$

When $S_{\text{clean}}(a) = 0$, ACRS is undefined rather than forced to zero. Per-family robustness is computed analogously:

$$\text{ACRS}_j(a) = \frac{S_j(a)}{S_{\text{clean}}(a)}.$$

The benchmark also reports absolute degradation $S_{\text{clean}}(a) - S_{\text{int}}(a)$ and relative degradation $1 - \text{ACRS}(a)$ when defined.

ACRS is intentionally simple and intentionally incomplete. It should never be reported without clean success, intervention success, sample size, uncertainty, and component metrics. An agent with low clean success can have undefined or misleading ACRS; an agent with equally poor clean and intervention performance can look superficially robust. The paper treats ACRS as a summary index, not as a sufficient statistic for agent competence.

Ranking instability. To test **Claim C4**, agents are ranked by clean success and by ACRS. The analysis reports rank deltas and Spearman correlation between rankings. A low correlation would indicate that clean success and causal robustness produce materially different model comparisons.

6 Experimental Setup

The planned main experiments use generated benchmark directories, deterministic seeds, fixed agent lists, and saved run metadata. Every run directory records the config, config hash, Python version, package version, timestamp, git commit when available, trajectories, errors, scores, aggregate summaries, and paper assets. The canonical commands are documented in `docs/RUNNING_EXPERIMENTS.md` and `docs/REPRODUCIBILITY.md`.

Agents. The initial package includes deterministic baseline agents: `RandomToolAgent`, `ScriptedOracleAgent`, `GreedyToolAgent`, `ReActStyleStubAgent`, and `PlannerExecutorStubAgent`. The oracle baseline uses hidden task metadata and is only an upper-bound sanity check; it must be excluded from realistic model-ranking claims. LLM-backed adapters are interfaces only until provider dependencies, prompts, model versions, API settings, and cost logging are finalized. A publishable experiment requires non-oracle LLM-backed agents, not only deterministic stubs.

Runs. Development artifacts currently exist under `results/20260510T110807Z_dev_20`. These artifacts are useful for testing the analysis code but are not used as final scientific evidence. The final paper must replace all bracketed values with runs over the frozen benchmark and audited agents.

Statistical reporting. The analysis layer exports bootstrap confidence intervals, paired clean-vs-intervention comparisons, per-family degradation, rank correlations, and effect-size summaries. These scripts are deterministic and generate placeholders when required experiment families have not been run.

7 Results

This section is a structured placeholder. No final scientific results are claimed in this draft.

7.1 RQ1: Does clean success overestimate robust competence?

Planned test. Compare clean success, intervention success, absolute degradation, relative degradation, and ACRS across agents. This tests **Claim C1**.

TODO. Insert Table 2 from the frozen main run. Current generated table template: `tables/table2_main_agent_performance`. Do not report the development-run values as paper findings.

7.2 RQ2: Which intervention families cause the largest degradation?

Planned test. Report success and degradation by intervention family, with representative trajectories for tool failure, memory corruption, observation conflict, and premature success signals. This tests **Claim C2**, **Claim C7**, and **Claim C8**.

TODO. Insert Figure 3 and Table 3 from the final run. Current generated templates: `figures/figure3_intervention_families` and `tables/table3_intervention_family_performance.*`.

7.3 RQ3: Do clean-success rankings match causal-robustness rankings?

Planned test. Compare agent rankings under clean success and ACRS, reporting Spearman correlation and rank deltas. This tests **Claim C4**.

TODO. Insert Figure 4 after main experiments. Current generated template: `figures/figure4_ranking_instability.*`.

7.4 RQ4: Do trajectory metrics expose failures hidden by final-answer scoring?

Planned test. Mine cases where final success is positive but trajectory success or component diagnostics indicate invalid tool use, missing evidence, blind memory trust, failed recovery, contradiction miss, or premature stopping. This tests **Claim C3**.

TODO. Insert Figure 6 and audited examples from `results/<run_dir>/error_cases/`.

7.5 RQ5: Can prompting or planning scaffolds improve robustness?

Planned test. Compare baseline agents or LLM prompts with and without self-checking, planning scaffolds, tool-verification instructions, and recovery prompts. This tests **Claim C6**.

TODO. Run ablations and replace the placeholder Table 4. The current table explicitly says “not yet run.”

8 Human Validation

Human validation is planned but not yet complete. The validation protocol should sample clean and intervention instances across domains, difficulties, intervention families, agents, and outcome types. Annotators or expert auditors should judge whether the task is understandable, whether the intervention preserves the high-level user goal, whether the changed factor is isolated, whether final-answer labels are correct, and whether trajectory diagnostics reflect real failures.

The paper should report agreement, adjudication procedure, annotator qualifications, compensation, and any systematic disagreements. This section is required before moving **Claim C3** or **Claim C10** from planned to supported. Table 5 is currently a placeholder and must remain so until validation is run.

9 Ablations

The planned ablations evaluate whether simple scaffolds improve causal robustness: explicit self-checking, planner-executor prompting, uncertainty-aware final answers, memory verification instructions, and recovery-specific tool-use policies. The analysis should report improvements, regressions, cost, latency, tool overuse, and family-specific effects.

No ablation result is claimed in this draft. This section exists to prevent the paper from over-attributing robustness failures to model architecture when prompt or scaffold changes might explain part of the effect. The planned evidence is tracked by **Claim C6**.

10 Limitations

Synthetic environment. The default benchmark uses synthetic tasks and deterministic simulated tools. This supports causal control and reproducibility, but may not capture the ambiguity, latency, partial observability, interface drift, and unexpected failure modes of real applications.

Gap to real-world agents. Performance on CAUSALAGENTBENCH may not predict deployment behavior for agents connected to live web, email, calendars, codebases, or enterprise tools. The benchmark should be viewed as a controlled diagnostic, not a substitute for domain-specific safety evaluation.

Scoring limitations. The default scorer is deterministic and auditable, but heuristic. It may miss semantically correct answers, reward template-specific wording, or misclassify trajectory faithfulness. Human validation and grader sensitivity analysis are required before strong empirical claims.

LLM-as-judge limitations. If LLM-as-judge scoring is introduced, results may depend on judge model, prompt, sampling settings, and provider version. Judge bias and instability must be measured rather than assumed away.

Intervention realism. Interventions may be too artificial, too obvious, or may change multiple factors despite the design goal. The single-factor-change assumption requires audit evidence, tracked by **Claim C10**.

Benchmark overfitting. Public tasks, tool schemas, and intervention patterns could be memorized or gamed. Future releases should include held-out templates, versioned splits, and reporting requirements for training contamination.

Cost and API reproducibility. Main LLM experiments may depend on paid APIs, changing model versions, rate limits, and provider-side nondeterminism. The repository should preserve prompts, model identifiers, seeds where supported, costs, and timestamps.

Human validation limits. Human validation can improve label quality but introduces annotator subjectivity, sampling limits, and compensation obligations. Agreement statistics should be reported with the same caution as model metrics.

11 Ethics and Reproducibility

Data and actions. Default tasks use synthetic mock data. The simulated email tool creates drafts only; it never sends real email. The booking tool is a stub and never makes real reservations. The benchmark should not include private personal data, API keys, proprietary documents, or live credentials.

Responsible release. CAUSALAGENTBENCH can be used to diagnose robustness failures, but it can also be gamed if treated as a static leaderboard. Releases should be versioned, include held-out components when possible, and distinguish oracle sanity checks from realistic agents.

Human subjects. If human validation is used, annotators should receive clear instructions, fair compensation, consent information, and a process for reporting ambiguous or uncomfortable examples. The paper must report the validation protocol and agreement statistics.

Transparency. Every reported result should include task version, run config, random seed, model/API version, timestamp, scorer version, and artifact path. Current reproducibility conventions are documented in `docs/REPRODUCIBILITY.md`. The smoke-test reproducibility claim is tracked by **Claim C9**; broader scientific claims remain planned until the main experiments are complete.

12 Conclusion

CAUSALAGENTBENCH is designed to evaluate not only whether tool-using agents succeed, but which skills survive targeted perturbation. The paper’s central hypothesis is that final task success can overestimate robust agent competence by conflating planning, tool use, memory, observation interpretation, recovery, and stopping behavior. The benchmark operationalizes this hypothesis through paired clean and intervention instances, ACRS, and trajectory-level diagnostics.

This draft does not yet claim final empirical support. The next milestone is to run the frozen benchmark with realistic LLM-backed agents, complete human validation, update the claim ledger, and replace all bracketed placeholders with reproducible evidence.

NeurIPS Checklist Draft

1. **Claims.** All scientific claims are tracked in `docs/CLAIM_LEDGER.md`. Unsupported claims are marked planned or placeholder.
2. **Limitations.** Limitations are discussed in Section 10 and `docs/ETHICS_AND_LIMITATIONS.md`.
3. **Reproducibility.** Code, configs, schemas, generated data, trajectories, score scripts, analysis scripts, and run metadata are part of the repository scaffold. Final reported runs must include artifact paths.
4. **Data.** Default data is synthetic. Dataset and benchmark cards document intended use, construction, fields, metrics, limitations, licensing TODOs, and maintenance plans.
5. **Human validation.** Not yet run. Required before strong claims about label quality or intervention validity.
6. **Compute and cost.** Smoke tests do not require paid services. Main LLM experiments must report provider, model version, date, sampling settings, cost, and latency.
7. **Ethics.** Default tools do not send real email, make bookings, or access private data.

References

- [1] Todo: Add agent benchmark survey citation, . To be completed after literature review.
- [2] Todo: Add software engineering agent benchmark citations, . To be completed after literature review.
- [3] Todo: Add tool-use benchmark citations, . To be completed after literature review.
- [4] Todo: Add web and navigation benchmark citations, . To be completed after literature review.